

RESEARCH ARTICLE

Counterfactual and Prototypical Explanations for Tabular Data via Interpretable Latent Space

SIMONE PIAGGESI¹, FRANCESCO BODRIA², RICCARDO GUIDOTTI^{1,3},
FOSCA GIANNOTTI², AND DINO PEDRESCHI¹

¹Department of Computer Science, University of Pisa, 56126 Pisa, Italy

²Scuola Normale Superiore, 56126 Pisa, Italy

³ISTI-CNR, 56127 Pisa, Italy

Corresponding author: Simone Piaggese (simone.piaggese@di.unipi.it)

This work was supported in part by the European Community Horizon 2020 Program under the funding schemes: SoBigData++ under Grant 871042, in part by HumanE Artificial Intelligence (AI) Net under Grant 952026, in part by European Research Council - Advanced Grant (ERC-ADG)-2018-ADG eXplainable Artificial Intelligence (XAI): Science and Technology for the eXplanation of AI Decision Making under Grant 834756, in part by the Foundations of Trustworthy AI – Integrating Reasoning, Learning and Optimization (TAILOR) under Grant 952215, in part by Social Explainable Artificial Intelligence (SAI) under Grant CHIST-ERA-19-XAI-010, in part by the Italian Ministry of University and Research (MUR) (N. not yet available), in part by Austrian Science Fund (FWF) under Grant I 5205, in part by the Engineering and Physical Sciences Research Council (EPSRC) under Grant EP/V055712/1, in part by National Science Centre (Poland) (NCN) under Grant 2020/02/Y/ST6/00064, in part by ETAg under Grant SLTAT21096, in part by Bulgarian National Science Fund (BNSF) under Grant KP-06-AOO2/5, in part by the NextGenerationEU Program under the funding schemes: National Recovery and Resilience Plan - Partenariati Estesi (Italy) (PNRR-PE)-AI (investment 1.3, line on AI) Future Artificial Intelligence Research (FAIR) under Grant M4C2, in part by SoBigData.it-Strengthening the Italian Research Infrastructure (RI) for Social Mining and Big Data Analytics-Prot. under Grant IR0000013, and in part by the Italian Project Fondo Italiano per la Scienza Mining Interpretable Models exploiting Sophisticated Algorithms (MIMOSA) under Grant FIS00001966.

ABSTRACT Artificial Intelligence decision-making systems have dramatically increased their predictive power in recent years, beating humans in many different specific tasks. However, with increased performance has come an increase in the complexity of the black-box models adopted by the AI systems, making them entirely obscure for the decision process adopted. Explainable AI is a field that seeks to make AI decisions more transparent by producing explanations. In this paper, we propose CP-ILS, a comprehensive interpretable feature reduction method for tabular data capable of generating Counterfactual and Prototypical post-hoc explanations using an Interpretable Latent Space. CP-ILS optimizes a transparent feature space whose similarity and linearity properties enable the easy extraction of local and global explanations for any pre-trained black-box model, in the form of counterfactual/prototype pairs. We evaluated the effectiveness of the created latent space by showing its capability to preserve pair-wise similarities like well-known dimensionality reduction techniques. Moreover, we assessed the quality of counterfactuals and prototypes generated with CP-ILS against state-of-the-art explainers, demonstrating that our approach obtains more robust, plausible, and accurate explanations than its competitors under most experimental conditions.

INDEX TERMS Learning latent representations, learning linear models, neural networks, counterfactual search, prototype search.

I. INTRODUCTION

Artificial Intelligence (AI) has become an integral part of every activity in our everyday life. Many companies have approached AI recently, from movie suggestions to autonomous vehicles [1], [2]. However, as we start using AI models for increasingly complex tasks, the decisions made become increasingly opaque, making the model a de

facto “black-box”. The higher transparency we can get into the AI black-box, the easier for people to trust it, and the more likely they will use it [3]. This thought gave rise to the field of eXplainable AI (XAI), an ever-increasing set of methodologies born to help humans understand the outcomes of AI algorithms. Moreover, explainability in AI has become one of the most important ethical and legal values identified by EU regulations (GDPR, AI act). In this work, we mainly consider the problem of explaining the predictions of Machine Learning (ML) black-box classifiers [4], [5].

The associate editor coordinating the review of this manuscript and approving it for publication was Xiong Luo¹.

There are several ways for generating explanations from black-box models [6], here we focus on counterfactuals and prototypes. These are *instance-based* explanations [4] because they consist in presenting to the user a set of records that help to understand the black-box behavior. They differ from *rule-based* explanations [7], which involve deriving rules that describe the decision process of the black-box classifier.

Counterfactual explanations are based on *contrastative* reasoning [8], a concept in psychology that involves the human tendency to create possible alternatives to events that have already occurred. Specifically, counterfactuals - or *counter-exemplars* - are defined as data records as close as possible to the original instance, but with an altered black-box outcome. Therefore, counterfactuals are intended to primarily satisfy the properties of data *Proximity* and prediction *Validity* [9]. Counterfactual explanations can be seen as a special case of adversarial examples [10], namely attacks designed to fool the model's prediction by presenting slightly perturbed instances. Unlike counterfactual explanations, where feature modifications aim to be reliable and insightful about the decision-making process, the changes in adversarial examples are intended to be imperceptible [11], making them useless for explanations. Thus, adversarial examples in data might be confused with possible counterfactual explanations, but they fail to meet two critical properties, i.e., *Plausibility* and *Robustness* [12]. Specifically, (i) adversarial examples are not plausible instances because they represent unrealistic data points, and (ii) adversarial examples are not robust because they typically cease to be valid with additional perturbations. This problem is highlighted on the top panel of Figure 1, where two adversarial examples *C* and *F* are selected as counterfactual explanations for data points *B* and *E*. These adversarial examples are inadequate as counterfactuals due to their location in low-density regions, outside the distribution of similarly labeled examples. This misplacement fails to accurately represent their predicted class, making them both implausible and non-robust as explanations.

Prototype explanations rely on the possibility of analyzing black-box behavior by comparison between analogous records, which is another type of reasoning widely used by humans [13]. In particular, prototypes are data records highlighting the characteristics that identify a group of objects belonging to the same predicted class. The membership of an instance to the category is determined by its similarity to the prototypes of the category. By looking at the similarity, it is possible to trace the main decisions of a black-box model by comparison with other instances. Prototypes serve as *examples* to understand the black-box reasoning, and consequently can be considered as counterfactuals' opposite. Bien et al. [14] proposed that a desirable prototype set for a given class should meet the following criteria: (i) covering as many training points of the same class as possible, (ii) covering as few training points as possible of different classes, and (iii) using the fewest possible

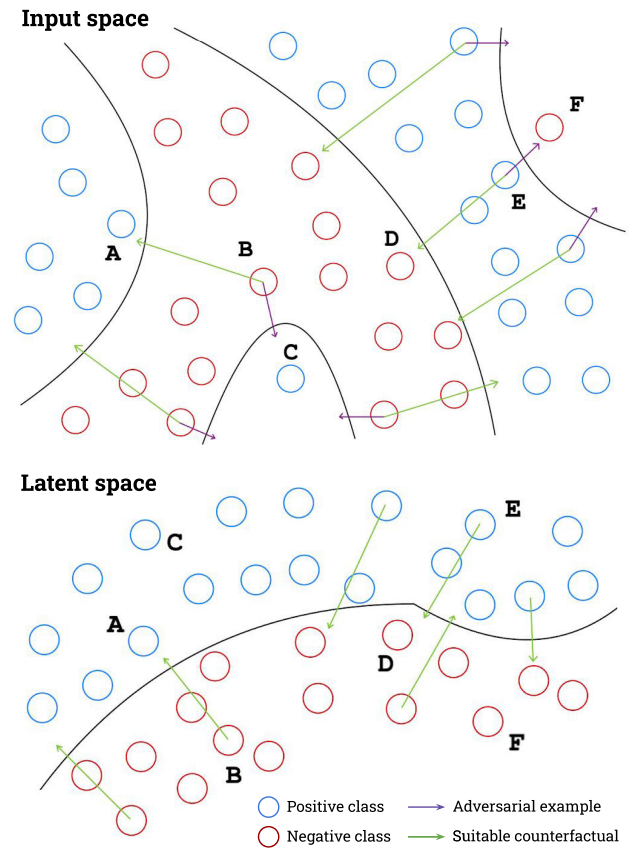


FIGURE 1. Illustration of an adversarial problem in counterfactual explanations and the proposed solution with CP-ILS. (Top) Given the input samples *B* and *E* in the original data space, the adversarial examples *C* and *F* are the closest instances that change the model's decision and might be mistaken for acceptable counterfactuals. These examples lie in low-density regions that are not representative of the predicted class, limiting the efficacy of the explanation. In contrast, instances *A* and *D* are located in dense regions of examples with a different predicted class, resulting in more robust and plausible counterfactuals for *B* and *E*. (Bottom) By learning the latent space according to feature and prediction similarities, the previously adversarial points *C* and *F* are no longer retrieved as good counterfactuals because they are not among the closest points to the decision boundary for *B* and *E*, respectively. In contrast, instances *A* and *D* remain robust and plausible counter-exemplars, as they are in the input space.

prototypes. In essence, a high-quality set of prototypical explanations should capture the underlying structure of the training instances for the target class, while also accounting for the structure of other classes, using the minimal number of prototypes.

Our objective is to exploit a *latent space representation* of tabular data to enable searching counterfactual and prototypical explanations that are effective and feasible for practical usages. To achieve this, we aim to construct the latent space incorporating similarity information of data points and their predictions from an external black-box model. As depicted in the bottom panel of Figure 1, creating a latent space using also black-box predictions allows the discovery of less adversarial counter-examples. In fact, by rearranging data according to the similarity of both features and black-box outcomes, adversarial examples like *C* and *F* are no longer feasible counterfactuals. This is because latent proximity is

influenced by the shape of the decision boundary, ensuring that the latent features are consistent with the black-box's logic. This alignment significantly reduces the chance of generating implausible counterfactuals, making the right counter-examples *A* and *D* the more feasible candidates.

The origin of latent space representations relies on dimensionality reduction techniques and representation learning [15], which have been traditionally applied in discriminative tasks [16]. Nowadays, latent transformations are the key components for encoder-decoder architectures, widely used in generative models [17], [18]. To create synthetic data according to input distribution, these generative models encode a reduced space, called latent space, in which data are positioned based on their similarity [19]. Placing data based on similarity makes it easier to measure distances and categorize data into groups, therefore allowing latent representation of data to find effective prototype explanations. Besides that, the generative capabilities of these models enable the fit of the underlying data distribution, helping the search of counterfactual instances that closely mimic the original data [17], [20]. However, explanations obtained from latent space representations are currently limited in several aspects. Firstly, the mapping from input features to latent space is typically performed by Neural Networks (NNs), which introduce multiple nonlinear transformations. This complexity renders the relationships between latent space dimensions and real features incomprehensible to humans [21]. Secondly, The creation of latent spaces is usually unsupervised, providing no guidance on how to navigate the space to change the prediction label of an instance [22]. Thirdly, most explanation methods leveraging latent spaces obtained through NN models are designed for image data [23], but since NNs often struggle with tabular data, these approaches become impractical for real-world applications like healthcare, banking, or credit scoring, where continuous and ordinal data are prevalent.

Therefore, it is crucial to confide in reliable explainers for tabular data. In this work, we present a method for generating Counterfactual and Prototypical explanations via Interpretable Latent Space (CP-ILS), specifically tailored for tabular data. CP-ILS builds a transparent latent space by using a linear approach and taking advantage of the predictions of a given black-box model. Thus, exploiting the transparency of the obtained latent representations, CP-ILS produces local and global post-hoc explanations able to overcome the problems highlighted above. Local explanations are obtained in the form of counterfactual instances, following the prediction direction of the black-box in the latent space, while global explanations are achieved as prototype search in this custom-designed space. The counterfactual explanations are decoded back into the real space using the linear nature of the transformation, making the whole process transparent. Previous works [24], [25] have moved towards this pathway, and our research further advances the direction from multiple perspectives. We have designed the counterfactual generation algorithm to allow the selection of features to change, thereby

enabling the actionability of the identified counterfactuals. The user can select which input data features wants to modify, and CP-ILS will produce a counterfactual with only that subset of features changed, if possible. Additionally, we have validated counterfactual results to assess the robustness and plausibility of the explanations, measuring their resemblance to adversarial attacks. Our experiments demonstrate that it is possible to use the interpretable latent space representations of CP-ILS to generate reliable explanations for tabular data, obtaining more robust and plausible counterfactuals than current explanation methods taken individually. Moreover, we have introduced the generation of global explanations as exemplar prototypes, further enriching the explanation output in the form of Counterfactual/Prototype pairs, which provide different views for understanding the black-box behavior. The results show that CP-ILS outperforms several state-of-the-art counterfactual and prototype explanation methods regarding multiple quantitative metrics. We focus our analysis on tabular data. However, we also provide promising experimental results for other data-types such as time series and text in Appendix III. This work introduces CP-ILS, a comprehensive framework for generating counterfactual and prototypical explanations by learning an interpretable latent space. Contributions of this paper are summarized as follows:

- 1) We present a novel technique for mapping tabular data into a projected space, where latent features are also aware of black-box predictions. This method ensures transparency through a linear relationship between input features and the learned latent directions.
- 2) We propose two algorithms for counterfactual and prototype search based on the latent linear model mentioned above, enabling the computation of both local and global explanations.
- 3) We conduct extensive experimental analyses to validate the proposed approaches, evaluating the quality of the resulting latent space, and the effectiveness of the generated explanations (both counterfactuals and prototypes).

The rest of the paper is organized as follows. In Section II, we analyze related works in latent space learning, counterfactual and prototype explanations. After that, Section III illustrates the proposed methodology. Then, Section IV reports the experimental results comparing our proposal against state-of-the-art methods for dimensionality reduction, counterfactual, and prototype explanation. Finally, conclusions and future research directions are discussed in Section V. Our code for testing the proposed algorithms is publicly available online.¹

II. RELATED WORKS

This section presents the most important studies concerning latent space methods, as well as the works for producing counterfactual and prototype explanations.

¹<https://www.github.com/simonepiaggese/cp-ils>

A. LATENT SPACE MODELS

When it comes to compressing high-dimensional data, there are two main categories of algorithms capable of satisfying the task: dimensionality reduction and generative models. Dimensionality reduction methods transform input features into a lower-dimensional space by preserving meaningful properties of the original data, while generative models learn the underlying distribution of data manifolds with the aim of generating new samples. Both of these types of models try to incorporate data information into a reduced latent space.²

1) DIMENSIONALITY REDUCTION

Dimensionality reduction techniques are unsupervised approaches that can reproduce the original space in a reduced latent space that preserves the data properties. Principle Component Analysis (PCA) [28] is the most famous one and it is defined as an orthogonal transformation that maps the data into a new coordinate system, reducing variance. t-Distributed Stochastic Neighbor Embedding (t-SNE) [29] overcomes PCA limitations in capturing local non-linear structures. As a variation of t-SNE, Uniform Manifold Approximation and Projection (UMAP) [30] uses graph layout algorithms to arrange data in low-dimensional space and preserve the local structure of the data. TriMap (TMAP) [31] tries to balance the importance of the local and the global structure of points in the created latent space using triplet constraints. However, the representations created by data reduction are opaque and hard to interpret, due to the highly non-linear structure of the majority of models. A new direction of research is to explore the possibility of using a linear model to understand latent space representations. Some preliminary works for images have been done in such direction [32] by using autoencoders combined with linear transformations.

2) GENERATIVE MODELS

The most common generative model is called Variational Auto-Encoder (VAE) [26]. A VAE is composed of an encoder, that maps the data into a latent space, and a decoder that reconstructs the data back into the original space. Both the encoder and the decoder functions are NNs. VAE improves the encoder-decoder representations by learning parameters of a probability distribution (usually Gaussian) over the latent variables, ensuring generation capabilities by sampling from this latent space. The training minimizes the reconstruction loss between the original data and the decoder-generated one, plus a regularization term that ensures the learned latent distribution remains close to the Gaussian unit. Several variations have been proposed to improve data representations, constraining it by distribution [26] or by predictions [33]. The majority of generative model works with image data [34]. On the other hand, designing and retrieving a latent space for tabular data is a task more

complex since the encoder and the decoder of the autoencoder are made by NNs, which are not optimal when using structured data. To deal with this type of data, new variations of VAEs have been created [35], [36]. Other works focus on incorporating causal relationships between variables in latent space representations [37], [38]. Another significant problem with the generative models approach is that since the encoder and decoder are NNs, they themselves can be considered black-boxes [39].

B. COUNTERFACTUAL EXPLANATIONS

In the field of XAI, a counterfactual explanation describes the modifications made to an instance to change its prediction [40]. Recently, this type of explanation has become very popular due to its ease of understanding by users, and many new methods of explanation have been developed in this direction [9].

1) COUNTERFACTUAL SEARCH METHODS

The core idea for finding counterfactuals is to set the search for such explanations as an optimization problem. The search strategy is called optimization searching since counterfactuals can be found by minimizing a proximity loss. One of the most famous works in counterfactual search is the one by Wachter et al [41], where such loss is composed by a quadratic distance between the desired outcome and the black-box prediction and the distance between the original sample and the counterfactual. DiCE [42] is based on recent research that generates counterfactual explanations for any ML model. The crucial difference is that explanations require perturbations to change the output of an AI model to be diverse and feasible to apply. A different strategy to create counterfactuals is to use a heuristic search strategy that aims at finding counterfactuals through heuristic choices that minimize a particular cost function at each iteration. Growing Spheres Generation (GSG) [43] relies on growing a sphere of synthetic instances around the original sample to find the closest counterfactual. The search is performed randomly in all possible directions until the decision boundary is crossed. Other approaches involve the use of a genetic algorithms [12], [44], Shapley values [45], or regression based searches [46]. For instance, it is worth mentioning LORE [44], a local model-agnostic explainer that provides explanations in the form of factual and counterfactual rules. It exploits a genetic algorithm to create a synthetic neighborhood of the instance to explain, and it trains a local decision-tree classifier to extract counterfactual rules.

2) COUNTERFACTUAL SEARCH IN LATENT SPACES

Integrating latent space into the counterfactual search can be done in two ways: by adding an additional term in the loss used for the search or by performing the search directly in latent space [47], [48]. The data are usually encoded in a latent space, then it is performed the search for a point that is considered a counterfactual. After that, the retrieved

²Here we refer specifically to generative models based on latent variables, such as VAEs [26] and GANs [27].

instance is decoded back into the original feature space using the decoder, and a check is done to determine if it results in a different prediction. This process is facilitated by the fact that the latent space is an Euclidean space, and every point in this space can be decoded back easily into the original space using the decoder. Most of the works using this procedure focus on images. The encoding of data in the latent space can be computed in different ways. In [49] it is used a median absolute deviation as a distance measure while in [50], they minimized the Fisher information. In ABELE [51], similarly to LORE, it is used a genetic algorithm to generate a neighborhood, but in the latent space, and a decision tree is trained on latent synthetic data to extract counterfactual rules.

3) COUNTERFACTUAL METRICS

When searching for a counterfactual example, it must be close to the original instance but with a different black-box outcome, satisfying criteria of *Proximity* and prediction *Validity*. This searching strategy is called optimization searching since we search counterfactuals by minimizing a loss [9]. In [41] such loss is composed by a quadratic distance between the desired outcome and the black-box prediction and the distance between the original sample and the counterfactual. In recent years, different losses were tested to ensure also the *Plausibility*, *Robustness*, and *Diversity* of the found counterfactuals. A plausible counterfactual means that its characteristics are similar to those of the original dataset [52]. For example, plausible counterfactuals should not be labeled as outliers with respect to the data [53]. Similarly, a counterfactual is considered robust if small feature perturbations still yield a valid counterfactual [54], [55]. This concept of *Robustness* is closely related to the *Actionability*, as a non-robust counterfactual can lead to issues when implementing practical recourse actions [56]. Finally, counterfactual methods should also produce several counterfactuals, as different as possible from each other. We refer to this criterion as the *Diversity* [42]. All these criteria help to quantify the feasibility of the proposed counterfactuals.

C. PROTOTYPICAL EXPLANATIONS

Regarding prototype explanations, there are two types of approaches: supervised and unsupervised ones. Unsupervised approaches look for a set of points in the dataset that help summarize its characteristics, while supervised approaches use label or prediction information to find a prototype set that explains the data or black-box behavior.

1) PROTOTYPE SEARCH METHODS

Kim et al. [57] developed an unsupervised algorithm for automatic prototype and critical search for a dataset, called MMD-critic. The algorithm tries to solve the disadvantage of overgeneralizing prototypes. In other words, the properties of prototypical members are assumed to be universally

shared by the group, even if substantial variation exists within the group. MMD-critic avoids overgeneralization by also showing exceptions or criticisms of the rule: minority data that differ substantially from the prototype but still belong to the category. Hase et al. [58] created a taxonomy using hierarchically organized prototypes that can be used for classification purposes. Bien and Tibshirani [14] have proposed ProtoSelect, which is a prototype selection method to build a condensed view of the data and also an interpretable model. ProtoSelect is designed to output prototypes with a set of desirable properties, which cover as well as possible the categories in the data with as few prototypes as possible. Future predictions can be made simply by looking at the classification of the most similar prototype. ProtoDash [59] is a supervised method to find similar data points that can help justify the behavior of the model to domain experts. By using non-negative weights, ProtoDash is able to best summarize the data and to compactly represent a complex data distribution in a set of prototypical samples. Other prototype methods are more constrained to the data type. Ming et al. [60] proposed ProSeNet: an interpretable deep sequence model with natural explanations derived from case-based reasoning. Chen et al. [61] introduced ProtoPNet, a deep learning network that can explain how it distinguishes birds and can extract sub-patches of images that most influenced its decisions. The versatility of prototypes makes them useful not only in the field of explanation but also in others. Since prototypes summarize the dataset, they can be used for zero-shot learning [62] or for one-shot learning [63]. This ability to generalize the information of a dataset can be also employed for counterfactuals generations [47] or for new data generation in generative models [64].

2) PROTOTYPE SEARCH IN LATENT SPACES

The ability of the latent space to generalize in a compressed space the properties of a dataset has been used in recent years to search for prototypes. The most common way to generate prototypes in the latent space is by doing clustering [65], but other approaches have shown that it is possible to use energy-based optimization methods [66]. Many applications have been developed in recent years that use prototypes created in the latent space. Liu et al. [67] showed that it is possible to use the latent space of a generative model to build prototypes and then use them to detect deviations from them in order to catch anomalies. A similar concept is used in Abati et al. [68], where this time the prototypes found in the latent space are used to detect novelties instead of anomalies. Other uses of the latent space prototypes are for face identification [69], guided generation of sentences [70], and for other explanations generations like saliency maps [71].

III. PROPOSED METHODS

Here we describe the proposed framework for Counterfactual and Prototypical explanations via Interpretable Latent Space (CP-ILS). This section is divided into two main parts. The

Algorithm 1 *CP-ILS*($X, Y, \vec{x}, b, p, F, s, \epsilon$)

Input : X - data samples, Y - data labels, $\vec{x}$ - instance to explain, b - black-box

Parameters: p - nbr of prototypes to produce, F - feature set to modify, s - latent space starting step, ϵ - step increment

Output : Z - latent samples,
 C - set of counterfactual explanations,
 P - set of prototype explanations

```

 $W \leftarrow \text{LatentSpaceLearning}(X, Y, b);$            // learn model
 $\hat{Y} \leftarrow b(X);$                                // obtain black-box predictions
 $Z \leftarrow W^T \cdot [X, \hat{Y}];$  // compute latent space
 $C \leftarrow \text{CounterfactualSearch}(\vec{x}, W, b, F, s, \epsilon);$  // CFs
 $P \leftarrow \text{PrototypeSearch}(X, W, b, p);$  // prototypes
return  $Z, C, P;$ 

```

first section reports the methodology used to learn the latent space representations, and the second part focuses on how to exploit the latent space to produce the counterfactual and prototype explanations. The pseudo-code for the entire process is described in Algorithm 1.

In summary, in our proposal we generate a latent space with interpretable features, and exploit its transparency to generate counterfactuals that preserve the correctness of the similarity and plausibility of the latent space. Our approach significantly differs from existing methods for finding counterfactual explanations [23], [42], [49], [72]. Most methods operate in the input space of data [42], [44], whereas our method operates in a latent space, providing a more reliable and noise-free environment with meaningful factors encapsulated in latent features. Although some approaches utilize latent spaces [23], [50], they often neglect the interpretability of the space itself, focusing solely on the resulting explanations. In contrast, our method ensures that the latent space remains interpretable, allowing users to understand the linear relationships contributing to the mapping. Furthermore, we demonstrate that it is possible to constrain latent space representations to align with the predictions of an external black-box model. While this characteristic is inspired by conditional autoencoders [33], we introduce a novel way to leverage this information to create superior counterfactual explanations. Specifically, our method incorporates a mechanism to guide the counterfactual search towards regions that are consistent with the black-box's decision boundary, helping the counterfactual generation. Additionally, our work redefines the use of prototypes, extending their role beyond data description to also explain the behavior of black-box models. Prototypes in our framework serve as reference points in the latent space, providing clear and interpretable exemplar categories that help users understand how different inputs correlate with black-box behaviors.

A. TRANSPARENT LATENT SPACE

In this part, we describe how it is possible to create a latent space with interpretable features using a linear model.

1) DESIGN PRINCIPLES

CP-ILS is based on a linear transformation that enables the transparent mapping between the input and the latent features. We opted for this direction because linearity has been proven to be one of the most reliable properties to produce explanations [73], [74]. The idea is to create/learn a latent space by combining a similarity loss analogous to t-SNE [29] with the parameterized mapping of autoencoders. Also, we want this mapping to be transparent (i.e., linear) to retain the link between the latent and the real features, as well as to be able decoding easily from the latent space back into the original one.³ In addition, latent space is also created using predictions from a black-box b , augmenting data information. In our methods, we consider black-box models trained for binary classification, leaving for future work possible extensions to the multi-class case.

The concept of proximity between two data samples has been consistently seen in terms of the similarity of their features. However, we argue that augmenting input features with the predictions given by a black-box could help to learn meaningful latent representations. The prediction score (such as classifier's probability estimates or decision functions) adds information about the black-box decision boundary to the latent space representation, contributing to identify relevant features that alter the prediction. As illustrated in Figure 1, creating a latent space using black-box predictions allows less adversarial counter-examples to be found. The blue data point C in the original space (top) cannot be a good counterfactual example for instance B because it is an adversarial examples, lying in a low-density region of instances with different prediction label. On the other hand, in the latent space (bottom), the data are rearranged according to the similarity of both features and prediction outcome, allowing for more diverse and less contradictory counterfactuals.

2) MODEL OVERVIEW

Given an input dataset $X \subseteq \mathbb{R}^n$ with ground-truth binary labels $Y \in \{0, 1\}^n$ and black-box predictions $\hat{Y} = b(X) \in \mathbb{R}^n$, CP-ILS learns a linear transformation $\mathcal{M} : \mathbb{R}^{(n+1)} \rightarrow \mathbb{R}^k$, which maps the concatenation $[X, \hat{Y}]$ into a latent space $Z \subseteq \mathbb{R}^k$. We refer to $\vec{x} = [x_i]_{i=1..n}$ as an instance belonging to the input space $\mathbb{R}^n$, $\hat{y} = b(\vec{x})$ the corresponding black-box prediction score, $\vec{z}(\vec{x}) = W^T \cdot [\vec{x}, \hat{y}] = [z_l]_{l=1..k}$ its latent representation in $\mathbb{R}^k$, and $W \in \mathbb{R}^{(n+1) \times k}$ the weights of the transformation with entries $\{w_{il}\}$. For brevity, in the rest of the paper we will use interchangeably $\vec{z}(\vec{x})$ or $\mathcal{M}(\vec{x}, \hat{y})$, implying that the latent mapping always depends on both $\vec{x}$ and $b(\vec{x})$. Through linear transformations it is

³If we relax the hypothesis of linearity, then to decode back in the input space there is the need for a decoder.

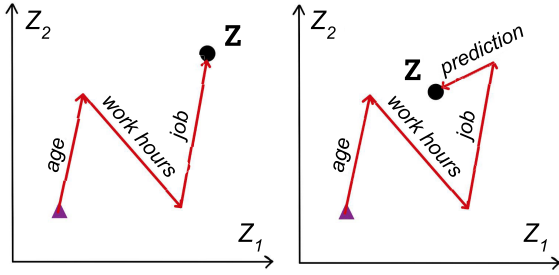


FIGURE 2. Representation of the contributions of the input features in the latent space. On the left, every latent point is decomposed as the sum of projections of input features onto the directions $\{\vec{w}_i\}$ defined by the rows of the transformation matrix. On the right, by concatenating the prediction of the black-box to the input we obtain a prediction direction $\vec{w}_y$ along which we have similar attributes but different predictions.

possible to isolate the contribution of each feature to the prediction. In particular, each input feature x_i is responsible for a direction of movement in the latent space as shown in Figure 2 (left). The dot product of the model $\mathcal{M}$ can be decomposed into a sum of contributions which gives the latent position $\vec{z}$. Indeed, the contribution of an input feature x_i to a latent space can be seen as a vector of dimensions equal to the dimensions of the latent space. Every feature in the input space contributed to a vector of k dimensions, which, when all added together, sum up to the latent position $z_l = \sum_{i=1}^n w_{il}x_i + w_{y,l}\hat{y}$, where the rows $\vec{w}_i = [w_{il}]_{l=1\dots k}$ of the weight matrix W represent the latent directions with respect to features x_i . Since CP-ILS concatenates the prediction $\hat{y}$ of the black-box b to the input we obtain in the latent space an additional direction $\vec{w}_y$ which we call the *prediction direction* (i.e., the last row of W) as depicted in Figure 2 (right). Latent samples that lie alongside this direction have similar input features but different prediction labels. We will exploit this crucial characteristic for finding counterfactual examples, relying on the linearity of the latent space.

3) LATENT SPACE LEARNING

The pseudo-code for learning the interpretable latent space is illustrated in Algorithm 2. CP-ILS uses gradient optimization [75] to minimize an objective that encourages both feature and prediction similarity among latent samples. Inspired by t-SNE [29], the similarity between two instances $\vec{u}$ and $\vec{x}$ is the probability that $\vec{x}$ would pick $\vec{u}$ as its neighbor if neighbors were picked in proportion to their probability density under a Gaussian distribution centered on $\vec{x}$. We use a matrix format to represent pair-wise similarities, with entries:

$$S_d(\vec{x}, \vec{u}) = \frac{\exp(-d(\vec{x}, \vec{u})/2\sigma^2)}{\sum_{\vec{v} \in X \setminus \vec{x}} \exp(-d(\vec{x}, \vec{v})/2\sigma^2)}, \quad (1)$$

where $\vec{x}$ and $\vec{u}$ are two vectors for which we compute the similarity, σ is the variance of the Gaussian, and d the distance measure between $\vec{x}$ and $\vec{u}$.

Given that the latent space is a continuous space, CP-ILS computes the similarity between two latent records with the cosine distance [76] ($d_Z = d_{cos}$). However, using cosine

Algorithm 2 *LatentSpaceLearning*(X, Y, b)

Input : X - data samples, Y - data labels, b - black-box

Output : W - trained transformation model

```

 $n \leftarrow \text{features}(X);$  // compute features length
 $k \leftarrow 2;$  // init. latent space dimensions
 $ACC_1 \rightarrow 0;$  // init. latent space score
 $\hat{Y} \leftarrow b(X);$  // obtain the black-box predictions
 $S_X \leftarrow \text{PairwiseSimilarity}(X);$  // original sim. matrix
while  $k < n \wedge ACC_{k-1} < ACC_k$  do
     $W \leftarrow \text{init}();$  // initialize model weights
    while do // until the loss decreases
         $Z \leftarrow W^T \cdot [X, \hat{Y}];$  // latent rep.
         $S_Z \leftarrow \text{PairwiseSimilarity}(Z);$  // latent sim.
         $L \leftarrow KLD(S_X || S_Z);$  // compute KL-Div.
         $W \leftarrow \text{BackProp}(W, L);$  // update weights
        iterate until convergence;
     $Z \leftarrow W^T \cdot [X, \hat{Y}];$  // compute latent space
     $ACC_k \leftarrow KNN(Z, Y);$  // compute validation score
     $k \leftarrow k + 1;$ 
return  $W;$ 

```

distance also in the input space would not work properly in the presence of categorical features, which are common in tabular data. To solve this problem that is typical of many counterfactual explainers [9], CP-ILS computes the distance between the categorical and continuous input features using a mixed distance [44] d_{mix} composed of two additive terms: the normalized Hamming distance [77] for categorical features, and the cosine distance for continuous features. Then, the difference between prediction scores is added to take into account the black-box contribution. The three distances are computed separately and then added together:

$$d_X(\vec{x}, \vec{u}) = \left(\frac{h}{m}\right) \cdot d_{hamm}(\vec{x}, \vec{u}) + \left(\frac{m-h}{m}\right) \cdot d_{cos}(\vec{x}, \vec{u}) + |b(\vec{x}) - b(\vec{u})|^2 \equiv d_{mix}(\vec{x}, \vec{u}) + |b(\vec{x}) - b(\vec{u})|^2 \quad (2)$$

where h is the number of categorical features, and m is the total number of features. Finally, the loss function minimized at each iteration is the Kullback-Leibler divergence [78]:

$$KL(S_X || S_Z) = \frac{1}{|X|} \sum_{\substack{\vec{x} \in X \\ \vec{u} \in X \setminus \vec{x}}} S_X(\vec{x}, \vec{u}) \log \frac{S_X(\vec{x}, \vec{u})}{S_Z(\vec{z}(\vec{x}), \vec{z}(\vec{u}))}, \quad (3)$$

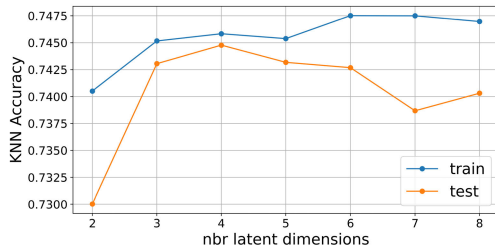


FIGURE 3. KNN accuracy score computed for different latent space dimensions for the *adult* dataset. As the dimensions increase, the KNN validation accuracy increases as well until we reach a maximum, where it starts decreasing. CP-ILS selects the last latent space dimension before the accuracy drop.

where S_X and S_Z are the similarity matrices⁴ computed respectively on the input space X and the latent space Z , with distances d_X and d_Z . The loss is back-propagated to update the weights W until convergence.⁵

When dealing with latent space representations, an essential parameter to select is the number of latent space dimensions k . The size k of the latent space is often chosen unwisely in the literature, usually based on the performance achieved in terms of reconstruction of the input [79]. Increasing the size of the latent space allows the space to store more information. This causes the creation of noisy dimensions, resulting in overfitting behavior. To avoid this effect, CP-ILS starts with the lowest possible dimension of latent space, i.e., $k = 2$, and progressively increases the dimensions until overfitting is detected. To identify possible overfitting behaviors, we test the validation accuracy of a KNN [80] classifier trained on top of the created latent space Z . In Figure 3 is depicted an example of such behavior.

Despite the commonalities, the optimization procedure differs from t-SNE for several reasons. Firstly, we used the same Gaussian similarity in the input and in the latent space (whereas in t-SNE latent distances are computed according to a Student-t distribution). Moreover, t-SNE runs an early exaggeration phase [81], specifically for enhancing the visualization of clusters, that do not help to learn a general-purpose latent space. Finally, training a parametric mapping function leads to more efficient embedding learning instead of directly optimizing the latent positions.

B. POST-HOC EXPLANATIONS

In this part, we describe how it is possible to use latent space representations learned by CP-ILS to search for counterfactual and prototypical explanations of the black-box b .

⁴To avoid memory overhead when computing similarity matrices, the fit is performed by dividing the data into fixed-sized batches and computing the similarity matrices separately for any batch of samples. This operation can be done only once for the input data, but it needs to be repeated every time for the latent space since the position of the points in Z changes after every iteration.

⁵For the convergence problem, we used the early stopping technique.

Algorithm 3 CounterfactualSearch($\vec{x}$, W , b , F , s , ϵ)

Input : $\vec{x}$ - sample to explain, W - latent space transformation, b - black-box
Parameters: F - feature set to modify, s - starting step, ϵ - step increment
Output : C - set of counterfactual explanations

```

 $\hat{y} \leftarrow b(\vec{x});$  // compute sample prediction
 $\vec{z} \leftarrow W^T \cdot [\vec{x}, \hat{y}];$  // compute latent space position
 $C \leftarrow \emptyset;$ 
for  $\tau \in \text{combinations}(F)$  do
     $\Delta \leftarrow \vec{0};$ 
    while  $b(\vec{x} + \Delta) = \hat{y}$  do // until same prediction
         $\vec{v} \leftarrow \vec{z} + s * \vec{w}_y;$  // set the destination
         $\Delta \leftarrow \text{Lagrange}(\vec{v}, \vec{x}, W, \tau);$  // optimize
         $s \leftarrow s + \epsilon;$  // compute new step
     $C \leftarrow C \cup \{\vec{x} + \Delta\};$ 
return  $C;$ 

```

1) COUNTERFACTUALS

For counterfactual explanations, we exploit the prediction direction found in the latent space to guide searching for better counter-examples. The procedure to find counterfactuals is illustrated in Algorithm 3. To enable the *Actionability* of the retrieved counterfactuals, the user can select a list of features F to modify and keep fixed other ones that are impossible to change, e.g., the age or the sex of a person. To determine the minimal changes Δ needed to transform an input data point $\vec{x}$ into a counterfactual $\vec{x}'$, we proceed as follows. First, CP-ILS encodes $\vec{x}$ into its latent space position $\vec{z} = W^T \cdot [\vec{x}, \hat{y}]$. Then, combinations of selected features F that most shift along the prediction direction are found. As illustrated on the left of Figure 4, the prediction direction is the violet arrow, while the effects of the real features on the latent space are the black vectors.

Specifically, our method iteratively moves from the encoded position $\vec{z}$ to new latent coordinates $\vec{v}$ with fixed incremental steps, $\vec{z} + s\vec{w}_y, \vec{z} + 2s\vec{w}_y, \dots$, towards the prediction direction. At each iteration, *Lagrange* optimization is performed to determine the minimal changes Δ that can be applied to the input $\vec{x}$ to modify its latent position to $\vec{v}$. This process continues until the black-box predicted label changes. On the right of Figure 4, the search is performed in three steps towards the prediction direction, from the starting position $\vec{z}$ until $\hat{y}$ changes. In Appendix I, we present mathematical and empirical evidence showing that an arbitrarily small change in the latent space results in a minimal perturbation even in the original data space.

To meet the *Diversity* propriety, desirable for a counterfactual method [42], we apply the search to different reduced combinations of features in F , namely for multiple subsets of features $\tau \in F$. Since it would be unfeasible to compute all the possible combinations due to exponential computing

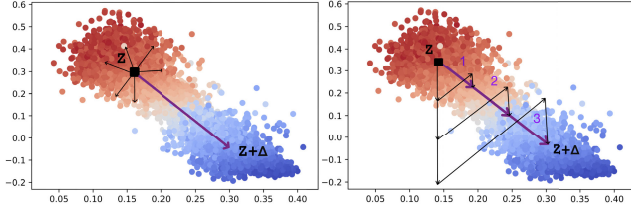


FIGURE 4. Visualization of the latent space created using two dimensions. The black square represents a sample $\vec{z}$ for which to look for counterfactuals. The violet arrows represent the path to follow, the black arrows are instead the effects of the input features on the latent space. On the left, we have the ideal combination of the feature to move towards the decision boundary, while on the right the same movement is obtained with the progressive search.

time, we only compute the search for a fixed number of features. This is reasonable since a counterfactual explanation with fewer features modified is better to comprehend from a human standpoint [9].

Given the linearity of the mapping, the Lagrange multipliers method [82] is used to optimize the squared norm of feature modifications $f = \|\Delta\|^2$ subject to constraints $\{g_l\}$:

$$\begin{cases} f = \frac{1}{2} \sum_{i \in \tau} \Delta_i^2 \\ g_l = \sum_{i=1}^{n+1} w_{il}(x_i + \Delta_i) - v_l \end{cases} \Rightarrow \begin{cases} \nabla f(\vec{x}) = \sum_{l=1}^k \lambda_l \nabla g_l(\vec{x}) \\ g_l(\vec{x}) = 0, \quad l = 1, \dots, k \end{cases}$$

where λ_l are the multipliers, and the constraints g_l are derived to ensure that the final latent position $\vec{v}$ matches the input feature perturbation $\vec{x} + \Delta$ after being mapped through W .⁶ By substituting f and g in the Lagrange equations on the above right, we obtain the linear system whose solutions are the minimal changes Δ that can be applied to the input $\vec{x}$ to modify its latent position to $\vec{v}$:

$$\begin{cases} \Delta_i = \sum_{r=1}^k \lambda_r w_{ir}, \quad i \in \tau \\ \sum_{i \in \tau} \sum_{r=1}^k \lambda_r w_{ir} w_{il} = v_l - \sum_{j=1}^{n+1} w_{jl} x_j, \quad l = 1, \dots, k \end{cases}$$

Besides user preferences, features to change can also be determined by different criteria, such as mutual information gain or feature relevance based on other local post-hoc methods [83]. The manual selection of the features to modify is a practical design choice, also adopted by other counterfactual methods. However, future work may explore more sophisticated approaches, such as automatically learning the optimal subset of features to modify based on specific objectives.

2) PROTOTYPES

For prototypical explanations, we aim to find clusters of instances with the same black-box prediction and compute

⁶To simplify the notation, in this system of equations we added an extra feature x_{n+1} to the instance $\vec{x}$ to identify the black-box prediction $b(\vec{x})$.

Algorithm 4 *PrototypeSearch*(X, W, b, p)

Input : X - training samples, W - latent space transformation, b - black-box

Parameters: p - nbr of prototypes to produce

Output : P - set of prototype explanations

$\hat{Y} \leftarrow b(X);$ // compute data prediction

$Z \leftarrow W^T \cdot [X, \hat{Y}];$ // compute latent space positions

$\Pi \leftarrow KMeans(Z, p);$ // obtain the centers

$P \leftarrow \emptyset;$

for $\vec{c} \in \Pi$ **do**

$\vec{z}(\vec{x}^*) \leftarrow \underset{\vec{z} \in Z}{\operatorname{argmin}} [d_{cos}(\vec{z}, \vec{c})];$ // find closest data

$P \leftarrow P \cup \{\vec{x}^*\};$

return $P;$

prototypes as the cluster centroids. The pseudo-code is illustrated in Algorithm 4. First, we obtain the black-box predictions $\hat{Y}$ of the training data X and its latent space representation $Z = W^T \cdot [X, \hat{Y}]$. Then, given that the latent space is composed entirely of continuous variables, we perform the arguably intuitive KMeans clustering [84] to find the set Π of latent centroids. Since the cluster centroids might not be latent representations of real instances, we identify the set of prototypes P by selecting the training instances whose latent embeddings are closest to the centers found. Namely, given a centroid $\vec{c} \in \Pi$, the corresponding prototype is the input instance $\vec{x}^* \in X$ such that $\vec{z}(\vec{x}^*) = \underset{\vec{z} \in Z}{\operatorname{argmin}} [d_{cos}(\vec{z}, \vec{c})]$.

The discovered prototypes provide a global explanation because they represent the entire training set of latent data points, rather than a single instance. For this reason, prototypes describe the distribution of input instances and the black-box behavior over the training set, making them suitable to characterize the decision-making process of the model from a global perspective.

Despite the inherently global nature of the prototypes described, a local prototype explanation can be found for a given input data sample $\vec{x}$ as the closest prototype $\vec{p}(\vec{x})$ to $\vec{x}$:

$$\vec{p}(\vec{x}) = \underset{\vec{p} \in P}{\operatorname{argmin}} d_{mix}(\vec{x}, \vec{p}) \quad (4)$$

where d_{mix} is used to account for both continuous and categorical features in the distance.

IV. EXPERIMENTS AND RESULTS

In this section, we report the experiments carried out to validate CP-ILS. First, we illustrate the datasets used, the classifiers, and the experimental setup. Then, we evaluated the goodness of the latent space created by CP-ILS, and the counterfactual and prototype explanations found in comparison with other methods. Finally, we comprehensively discuss the obtained results.

TABLE 1. Description of datasets and performance of the black-boxes.

	adult	compas	german	diva	fico	marketing	churn	wisconsin
nbr features	8	33	20	88	24	63	13	30
nbr instances	48842	6171	1000	8000	10459	4119	3150	569
XGB	.757	.956	.759	.927	.704	.913	.965	.947
RF	.763	.966	.778	.905	.711	.905	.944	.947
SVM	.763	.944	.777	.859	.711	.905	.902	.964
NN	.755	.943	.775	.864	.710	.899	.900	.939

A. EVALUATION SETUP

In all the experiments, for CP-ILS we learned the best latent space dimension with a batch size of 4096, Adam optimizer [75] with learning rate of 10^{-3} and early stopping of 3. In counterfactual search, we used $s = 0.1$ as the starting step and made an increase step ϵ of 0.1. The Lagrange multipliers system is solved using the `scipy` library.⁷

1) DATASETS AND BLACK-BOX MODELS

We ran experiments on a selection of tabular datasets widely referenced for classification tasks and publicly available from the UCI ML repository.⁸ We tested on *adult*, *compas*, *german*, *fico*, *marketing*, *churn*, and *wisconsin* datasets. In addition, we added a real-world proprietary dataset called *diva*.⁹ The datasets are split: 80% training, 10% validation, and 10% test set. As black-box models, we tested four different methods¹⁰ including XGBoost (XGB) [85], Random Forest (RF) [86], Support Vector Machines (SVM) [87], and dense fully connected Neural Networks (NN). We exclusively focus on the chosen class of models from the NNs family, avoiding the need to treat other neural architectures as black-boxes. Fully connected neural networks, being the most general class of deep learning classifiers, are a valid representative for neural models and align with our goal of embracing several types of input decision-making systems, being agnostic as much as possible. We performed five-fold stratified cross-validation for every black-box. Dataset descriptors and black-box performances are reported in Table 1. For a more concise presentation, in the next sections we will report experimental results for datasets *adult*, *compas*, *german* and *diva*. Specifically, the Tables 2-3-4 will showcase the experimental results using the most accurate black-box models for the selected datasets. Instead, Figures 5-6 will provide analyses with additional black-box models. The Tables with extensive results for all datasets and black-boxes are reported in Appendix II.

⁷<https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve.html>

⁸<https://archive.ics.uci.edu/ml/index.php>

⁹<https://kdd.isti.cnr.it/project/diva>

¹⁰XGB implemented as <https://xgboost.readthedocs.io/en/stable/>, RF and SVM from <https://scikit-learn.org/stable/>, NN implemented with `PyTorch` library <https://pytorch.org/>

2) ADAPTATION TO OTHER DATA-TYPES

Although CP-ILS is primarily designed for tabular data, it can be adapted to work with other data-types such as textual content or time series. Following pre-processing strategies like bag-of-words, an instance in the form of a time series or a sentence can be mapped into a vector with semantic attributes. Thus CP-ILS can be used on the resulting “tabular-like” representations, where every feature should have a clear and interpretable semantic, ensuring in this way the preservation of interpretability properties of the latent features. In Appendix III, we describe the procedure performed to validate CP-ILS on diverse data-types, and we report experimental results showing that the proposed model can effectively adapt to various data sources without appreciable loss of performances in latent space goodness and explanations quality. In the rest of the main paper, we focus on results obtained for tabular data.

B. INTERPRETABLE LATENT SPACE QUALITY

In this section, we evaluate the quality of the latent space created by CP-ILS. We test the goodness of latent spaces by examining the neighborhood structure preservation. Preserving pair-wise distances is important for two main reasons: firstly, it enables dimensionality reduction, offering a tool to encode high-dimensional datasets alongside black-box classification patterns; secondly, it ensures accurate mapping of distributional characteristics and cluster structures of input data, which will facilitate the subsequent extraction of high-quality explanations. In line with [88], we used different metrics described in the following.

1) METRICS

To evaluate the goodness of the latent space created, we measured the relative positioning of neighborhoods. We sampled random observations and computed the *Random Triplet Accuracy* [88], which is the percentage of triplets whose relative distance order is preserved in the high and low-dimensional spaces, the closer to 1, the better. Also, we measure the amount of outliers in the input space that remain outliers in the latent space. We call this property *Outlier Preservation*. We used the Isolation Forest algorithm [89] to measure which points are labeled outliers, running the algorithm in both spaces to check for changes. The percentage of the changes gives the final score: the lower, the better.

2) COMPETITORS

We compared CP-ILS against two different categories of algorithms: autoencoders and dimensionality reduction methods. Since CP-ILS is an hybrid approach between these two categories, we decided to include both in the experiments. The methods tested are PCA¹¹ [28], UMAP¹² [30], TMAP¹³ [31], ISOMAP¹⁴ [90] and VAE [26]. For VAE, we trained a 3-layers autoencoder with hidden dimensions equal to the number of input features divided by 2, for a maximum of 1000 epochs with early stopping 5, using Adam optimizer with learning rate 10^{-4} , and batch size 4096. For PCA, ISOMAP, UMAP and TRIMAP we used the standard parameters. We highlight that we did not compare against t-SNE as its main goal is to define a 2-dim space for visualization purposes rather than a latent space to perform further mining. Also, due to the $\mathcal{O}(|X|^2)$ time complexity of standard t-SNE, where $|X|$ is the dataset size, popular accelerate implementations are ineffective for our intents since they are specifically designed for 2-dim [91] or at most 3-dim [92] reduction problems.

3) RESULTS

For CP-ILS we used the KNN accuracy metrics described in Section III-A to select the best latent space dimensions. Since only CP-ILS has an internal criterion for choosing the best latent space dimension, for the other approaches we performed such selection by choosing the model configuration with the best task-specific score. We tested the latent dimensions from the list $D = \{2, 3, 4, 5, 7, 10, 15, 20, 25, 30\}$ to cover most of the possible dimensions while not exaggerating computational times. In Table 2 we report the results of the best latent space dimension according to space evaluation metrics. Table 2 (left) shows that CP-ILS is the best to preserve distances, with PCA and ISOMAP as second best approaches. The other methods largely fail to preserve the original distances. This is probably due to the fact that the preservation of the distance is not explicitly optimized. Table 2 (right) shows the results of the outlier preservation metric. Here we observe that PCA performs best in three out of four cases, with CP-ILS as the second-best approach overall. We conclude that our approach exhibits competitive performance when compared to the current state-of-the-art techniques in dimensionality reduction and autoencoders. Importantly, empirical evidence suggests that the linear nature of the latent space does not compromise the efficacy of our method, even when dealing with the complexity of real-world datasets.

C. COUNTERFACTUAL EXPLANATIONS EVALUATION

In this section, we describe the results obtained for the counterfactual explanations produced by CP-ILS. Evaluating

the quality of counterfactual instances is a complex task, as multiple factors must be considered to assess their effectiveness and usability [9]. In this study, we focus on *Proximity*, *Robustness*, *Plausibility*, and *Diversity*, which are among the most commonly tested properties in the literature for measuring counterfactual quality.

1) METRICS

Given the set $C(\vec{x})$ of counterfactuals found for the original sample $\vec{x}$, we express the evaluation metrics of the counterexamples $\vec{x}' \in C(\vec{x})$ as distances. For *Proximity*, *Robustness*, and *Plausibility*, the lower the distance values, the better are the counterfactuals returned; for *Diversity* measures, the higher the distances, the better the variety of counterfactuals. All the metrics are reported as average values over the test set of instances X_{test} for which we apply the counterfactual search. Some methods return more than one counterfactual; then we selected the best counterfactual among the set returned.

We measure *Proximity* in two different fashions [9], [42]. The first one, named d_{dist} , is the average distance between $\vec{x}$ and the counterfactual $\vec{x}'$, computed using a mixed distance defined in Equation 2:

$$d_{dist} = \frac{1}{|X_{test}|} \sum_{\vec{x} \in X_{test}} d_{mix}(\vec{x}, \vec{x}')$$

The second measure d_{count} , quantifies the average number of features changed between a counterfactual $\vec{x}'$ and $\vec{x}$, i.e.,

$$d_{count} = \frac{1}{|X_{test}|} \sum_{\vec{x} \in X_{test}} \frac{1}{n} \sum_{i=1}^n \mathbb{I}[x'_i \neq x_i]$$

where $\mathbb{I}$ returns 1 if $[condition]$ is true, 0 otherwise.

We measure *Plausibility* in terms of *Implausibility* [93] of the generated counterfactuals as the distance of $\vec{x}'$ from the closest instance in the training set X_{train} :

$$d_{impl} = \frac{1}{|X_{test}|} \sum_{\vec{x} \in X_{test}} \min_{\vec{u} \in X_{train}} d_{mix}(\vec{x}', \vec{u})$$

With respect to *Robustness*, we define d_{rob} to measure how much the predicted label of found counterfactual is similar to its neighbors in the reference population X_{train} . First, we sample the neighborhood of the selected counterfactual $N(\vec{x}')$ ¹⁵ from training instances, then we measure the average disagreement of the predicted classes:

$$d_{rob} = \frac{1}{|X_{test}|} \sum_{\vec{x} \in X_{test}} \frac{1}{|N|} \sum_{\vec{u} \in N(\vec{x}')} \mathbb{I}[y_b(\vec{u}) \neq y_b(\vec{x}')]]$$

where y_b is the predicted label of the black-box. The objective is to have a counterfactual $\vec{x}'$ close to the reference population (plausible) but near samples labeled opposite to the original sample $\vec{x}$ (robust). Besides, a counterfactual $\vec{x}'$ outside the

¹¹<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

¹²[urlhttps://github.com/lmcinnes/umap](https://github.com/lmcinnes/umap)

¹³<https://github.com/eamid/trimap>

¹⁴<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.Isomap.html>

¹⁵We used the 10 nearest training samples ($|N| = 10$) as neighborhood estimation.

TABLE 2. Space quality metrics. Best scores are in bold, second best result are underlined.

Dataset	Random Triplet Accuracy						Outlier Preservation					
	CP-ILS	VAE	PCA	UMAP	TMAP	ISOMAP	CP-ILS	VAE	PCA	UMAP	TMAP	ISOMAP
adult/SVM	.8657	.7093	.8026	.7189	.6093	.8083	<u>.0016</u>	.0028	.0002	.0026	.0019	.0073
compas/RF	.8235	.7325	<u>.8017</u>	.7096	.6105	.7257	.0009	.0016	<u>.0013</u>	<u>.0013</u>	.0014	.0022
german/RF	<u>.7982</u>	.7109	.8196	.7215	.6908	.7525	<u>.0012</u>	.0016	.0006	.0026	.0015	.0013
diva/XGB	.8531	.7215	.7593	.7122	.7286	<u>.7970</u>	<u>.0005</u>	.0006	.0003	.0013	.0007	.0072

training distribution or near instances of a wrong class will result more likely in an adversarial sample.

Moreover, we measure the *Diversity* in two fashions. The first one denoted as div_{dist} , measures the average pair-wise distance within the set of found counterfactuals $C(\vec{x})$. In contrast, div_{count} measures the average number of disagreeing features:

$$div_{dist} = \frac{1}{|C|^2} \sum_{\vec{x}' \in C} \sum_{\vec{x}'' \in C} d_{mix}(\vec{x}', \vec{x}'')$$

$$div_{count} = \frac{1}{|C|^2} \sum_{\vec{x}' \in C} \sum_{\vec{x}'' \in C} \frac{1}{n} \sum_{i=1}^n \mathbb{I}[x'_i \neq x''_i]$$

Finally, counterfactual methods might not succeed in producing valid counterfactual instances. For example, due to the optimization procedure, the counterfactual search might fail to find a feasible solution to the minimization problem. We computed the Success Rate (SR) of the various algorithms when searching for a counterfactual as the percentage of the successful counterfactual found with respect to the test set size:

$$SR = \frac{|\{\vec{x} \in X_{test} : C(\vec{x}) \neq \emptyset\}|}{|X_{test}|}.$$

This definition is intended to not penalize methods designed to return a single counterfactual.

2) COMPETITORS

We compare our proposal against different algorithms present in literature capable of producing counterfactual explanations. We selected the most popular methods that use different approaches to the problem with different focuses. For GSG [43], we used the suggested initial ratio of 0.1 with a factor of increase of 10, and for every radius, we generated 2000 samples and checked for counterfactuals. For DiCE [42], we generated four counterfactuals. In the WACH algorithm [41], we started with $\lambda = 0.1$ and then increased by 0.1 if the algorithm failed to find a counterfactual. For LORE [44], we exploit the genetic algorithm to generate 300 neighborhood samples. LORE generates counterfactual rules that indicate changes to the input that would lead to a different decision by the model. Given that, we took the counter-rules generated by LORE and used the thresholds given in the rule as indications to produce a counter-example. For instance, if the rule is: “the prediction would flip if feature A would be greater than 4.3”,

then we substitute the value of feature A, in the sample in analysis, with 4.3, and compute the metrics. For the sake of generality, we considered all the input features as actionable when running the different methods.

3) RESULTS

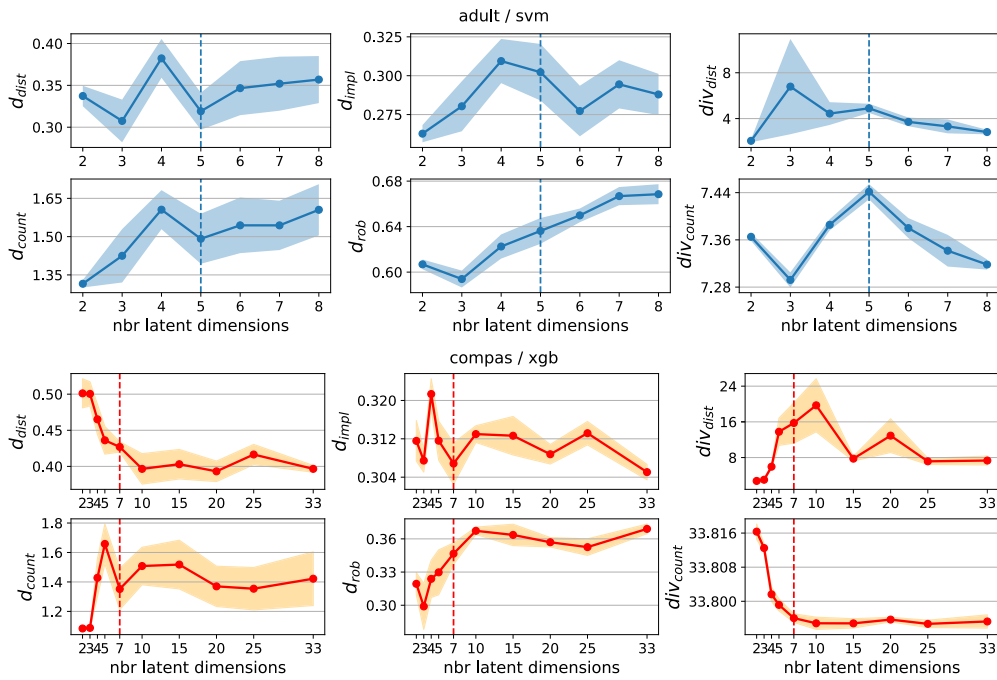
Table 3 reports the means and standard deviation for the counterfactual evaluation metrics. *Diversity* metrics are reported for CP-ILS and DiCE, the two methods capable of generating multiple counterfactuals per instance. CP-ILS comprehensively outperforms competitors across all datasets, particularly in d_{dist} and d_{count} metrics (*Proximity* measures). This observation confirms the CP-ILS’s ability to more effectively identify which features need modification to alter the black-box prediction, as shown in the previous section. Moreover, the transparent latent space search contributes to generating counterfactuals that closely resemble training set samples, being consistently among the best two methods in terms of *Plausibility* (d_{impl}). In addition, generally, the *Diversity* of the counterfactuals is better than the one of DiCE, concerning each of the metrics div_{dist} and div_{count} . Both CP-ILS and LORE generate effective explanations, with CP-ILS offering a slight edge in robustness d_{rob} and fewer feature changes d_{count} . Although CP-ILS is only marginally better than LORE in the d_{dist} metric, it proves to generate slightly more plausible counter-examples overall.

The combined evaluation for distance d_{dist} and robustness d_{rob} metrics requires a more nuanced discussion. Rarely does a method excel in both these metrics simultaneously. It only occurs for *adult*, where CP-ILS produces counterfactuals that are both similar and robust. The challenge in generating counterfactuals lies in balancing the distance from the original sample and the proximity to samples of different classes. Often, vectors closer to the original sample tend to be more adversarial, while those further away are less so. Despite this, CP-ILS consistently ranks first in one of these metrics and a close second in the other, showcasing its strong performance.

We further investigate the duality of being a close counterfactual without being adversarial by analyzing how the metrics for CP-ILS vary with changes in latent space dimensions. Figure 5 presents the results for the *adult* dataset using SVM and the *compas* dataset using XGB. Increasing the number of latent space dimensions adds flexibility in creating latent representations, leading to less compression of input feature information and more efficient

TABLE 3. Counterfactual metrics. The best results are highlighted in bold, second best results are underlined.

adult/SVM	d_{dist}	d_{count}	d_{impl}	d_{rob}	div_{dist}	div_{count}	SR
CP-ILS	0.244 $\pm$ 0.266	1.105 $\pm$ 1.025	0.193 $\pm$ 0.091	<u>0.653</u> $\pm$ 0.223	4.683 $\pm$ 0.631	4.440 $\pm$ 0.883	100%
GSG	0.835 $\pm$ 0.254	4.430 $\pm$ 1.283	0.673 $\pm$ 0.181	0.681 $\pm$ 0.291	-	-	100%
DiCE	1.377 $\pm$ 0.199	<u>2.105</u> $\pm$ 0.447	0.747 $\pm$ 0.521	0.691 $\pm$ 0.214	1.194 $\pm$ 0.170	1.809 $\pm$ 0.413	76%
WACH	2.492 $\pm$ 1.319	7.000 $\pm$ 0.000	0.992 $\pm$ 0.638	0.369 $\pm$ 0.354	-	-	88%
LORE	<u>0.264</u> $\pm$ 0.311	2.142 $\pm$ 1.040	0.274 $\pm$ 0.078	0.895 $\pm$ 0.256	-	-	100%
compas/RF	d_{dist}	d_{count}	d_{impl}	d_{rob}	div_{dist}	div_{count}	SR
CP-ILS	0.432 $\pm$ 0.372	1.800 $\pm$ 0.529	0.275 $\pm$ 0.187	0.393 $\pm$ 0.200	7.609 $\pm$ 13.262	6.596 $\pm$ 0.963	100%
GSG	0.447 $\pm$ 0.330	2.760 $\pm$ 4.495	0.397 $\pm$ 0.170	0.711 $\pm$ 0.137	-	-	100%
DiCE	0.713 $\pm$ 0.504	1.556 $\pm$ 0.497	0.206 $\pm$ 0.164	0.406 $\pm$ 0.208	0.976 $\pm$ 0.327	2.079 $\pm$ 0.585	27%
WACH	3.619 $\pm$ 0.330	26.217 $\pm$ 4.495	2.560 $\pm$ 0.147	0.117 $\pm$ 0.137	-	-	60%
LORE	<u>0.445</u> $\pm$ 0.419	1.814 $\pm$ 0.571	0.333 $\pm$ 0.151	0.394 $\pm$ 0.206	-	-	100%
german/RF	d_{dist}	d_{count}	d_{impl}	d_{rob}	div_{dist}	div_{count}	SR
CP-ILS	0.455 $\pm$ 1.317	2.130 $\pm$ 0.823	0.310 $\pm$ 1.059	0.479 $\pm$ 0.276	6.337 $\pm$ 5.483	5.111 $\pm$ 1.334	81%
GSG	0.687 $\pm$ 0.322	3.750 $\pm$ 2.334	0.357 $\pm$ 0.144	0.577 $\pm$ 0.220	-	-	81%
DiCE	<u>0.643</u> $\pm$ 0.659	4.676 $\pm$ 4.031	0.292 $\pm$ 0.239	0.578 $\pm$ 0.176	1.792 $\pm$ 0.687	5.552 $\pm$ 4.114	74%
WACH	1.250 $\pm$ 0.227	12.525 $\pm$ 6.177	0.366 $\pm$ 0.035	0.267 $\pm$ 0.047	-	-	29%
LORE	0.680 $\pm$ 0.512	19.200 $\pm$ 5.211	0.529 $\pm$ 0.427	<u>0.326</u> $\pm$ 0.223	-	-	100%
diva/XGB	d_{dist}	d_{count}	d_{impl}	d_{rob}	div_{dist}	div_{count}	SR
CP-ILS	0.085 $\pm$ 0.216	2.252 $\pm$ 0.433	0.403 $\pm$ 0.113	0.211 $\pm$ 0.139	0.291 $\pm$ 0.485	44.257 $\pm$ 1.457	100%
GSG	0.290 $\pm$ 0.595	35.623 $\pm$ 2.681	0.625 $\pm$ 0.453	0.769 $\pm$ 0.274	-	-	100%
DiCE	0.682 $\pm$ 0.349	9.578 $\pm$ 1.087	0.781 $\pm$ 0.504	0.608 $\pm$ 0.202	1.186 $\pm$ 0.475	11.550 $\pm$ 1.237	78%
WACH	2.756 $\pm$ 0.347	87.972 $\pm$ 0.009	1.864 $\pm$ 0.190	0.236 $\pm$ 0.011	-	-	72%
LORE	<u>0.155</u> $\pm$ 0.271	2.240 $\pm$ 0.509	0.288 $\pm$ 0.109	0.562 $\pm$ 0.199	-	-	100%

**FIGURE 5.** Average counterfactual metrics and standard errors in comparison with latent space dimension variations. On the top, we have the *adult* dataset with the SVM black-box. On the bottom, we have the *compas* dataset when using the XGB black-box. Vertical dashed lines correspond to the dimensions with best KNN validation accuracy.

data storage. However, this reduced compression can result in a less meaningful latent space, akin to overfitting behaviors. This effect is captured by the d_{rob} metric, which begins to increase after a certain dimension, eventually reaching a plateau. From Figure 5, it is evident that for the *adult* and *compas* datasets, the d_{rob} metric sharply increases when

using more than 3 latent dimensions. An interesting behavior occurs for div_{dist} metric: it initially increases at lower dimensions but experiences a significant drop at higher dimensions. In the *adult* dataset, CP-ILS selected 5 latent dimensions as optimal using the overfitting procedure described in Figure 3. This choice results in lower d_{count} and d_{dist} scores

and higher div_{count} scores. Regarding the `compas` dataset, lower dimensions tend to produce closer and more robust counterfactuals. CP-ILS selects 7 dimensions as optimal by measuring overfitting with a KNN classifier, adjusting the metrics d_{count} , d_{dist} , and d_{impl} . Although different dimensions optimize individual metrics—10 for div_{count} and d_{dist} , and 3 for div_{dist} and d_{rob} —7 dimensions offer the best overall performance across all metrics.

D. PROTOTYPE EXPLANATIONS EVALUATION

In this section, we present the results obtained for the prototype explanations produced by CP-ILS. High-quality prototypes enable the possibility of analyzing black-box behavior by comparisons between records, a common reasoning rationale used by humans. By examining records similar to the found prototypes, the user can gain insight into the black-box's decision reasoning. Therefore, we assess the quality of the prototypes by looking at how accurately they reflect input data and black-box outcome.

Metrics. To evaluate the goodness of the prototype explanations, we measured for every set of prototypes found by the different explainers, two types of metrics: the *1-KNN Accuracy* and the *Fidelity*.

We measure the *1-KNN Accuracy* of prototype explanations P using a 1-KNN classifier on the test set. The metric is defined as the accuracy of a 1-KNN classifier built on top of the prototypes found by the explainer. Given a test sample $\vec{x}$, and the closest prototype to the sample $\vec{p}(\vec{x})$ defined in Equation (4), the classification of the test sample is defined as the label of the most similar prototype:

$$1 - KNN\ Accuracy = \frac{1}{|X_{test}|} \sum_{\vec{x} \in X_{test}} \mathbb{I}[y_b(\vec{p}(\vec{x})) = y_{true}(\vec{x})]$$

where y_{true} are the ground-truth labels and y_b the predicted labels by the black-box. The closer to 1 the better.

We want a prototype to resemble the data it represents as much as possible, both in terms of similarity and black-box prediction. We called this last propriety *Fidelity*, and we defined it as the percentage of data with the same prediction as the closest prototype. More formally:

$$Fidelity = \frac{1}{|X_{test}|} \sum_{\vec{x} \in X_{test}} \mathbb{I}[y_b(\vec{p}(\vec{x})) = y_b(\vec{x})]$$

The closer to 1 the better.

1) COMPETITORS

We compared our proposal against different algorithms present in the literature presented in Section II capable of producing prototype explanations. We selected the most popular methods that use different approaches to the problem with different focuses. In particular, we tested our approach against MMD-Critic (MMD) [57], Proto-Select (P-Sel) [14], and Proto-DASH (P-DASH) [59]. These algorithms were designed to explain the label of the data rather than

predictions. In order to make a fair comparison with our approach, we used the prediction of the black-box rather than the labels in the searching phase of every method.

2) RESULTS

The results for prototype metrics are detailed in Table 4. A key parameter to fine-tune is the number of prototypes included in the final set of explanations. Selecting too few examples leads to an overgeneralized set of prototypes that would ignore some particular data subsets. On the other hand, selecting too many prototypes would lead to overfitting and producing redundant explanations. We report the best metrics derived from various prototype set dimensions chosen from the list $K = \{3, 4, 5, 6, 7, 8, 9, 10, 12, 14, 16, 18, 20\}$.

Our approach consistently outperforms competitors concerning the *1-KNN Accuracy* metric, except for the `german` dataset, where it achieves the second-highest performance behind MMD-critic. Notably, MMD-critic also ranks second in the `adult` and `diva` datasets. In terms of *Fidelity*, our method demonstrates clear superiority, achieving the highest scores in both the `german` and `diva` datasets. Regarding `adult` and `compas`, where MMD-critic and Proto-Select emerge as the best methods, CP-ILS ranks second. However, in the `compas` and `diva` datasets, MMD-critic is outperformed by Proto-Select. In these latter datasets, the higher performance of the black-boxes uncovers more complex patterns, making it more challenging for an unsupervised method like MMD-Critic to effectively identify a suitable set of prototypes.

Figure 6 illustrates the behavior of the two metrics as the number of prototypes increases. As expected, all methods show low performance for both metrics when the number of prototypes is small. Performance improves and reaches a plateau, indicating that the maximum amount of information has been captured. Our approach outperforms the other methods when using a number of prototypes greater than 10. For fewer than 10 prototypes, the metrics are more unstable, with no single method outperforming the others. We conclude that our approach is in line with the current state-of-the-art when summarizing data with a low number of prototypes, while it excels at identifying peculiar data groups when a larger number of prototypes is used.

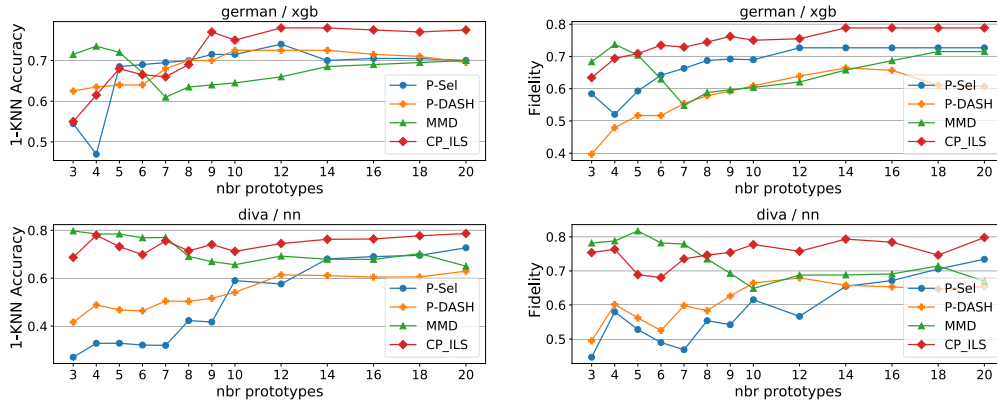
E. DISCUSSION

Experiments show promising results toward the construction of interpretable latent spaces for explaining black-box decision-making. Firstly, we compared CP-ILS with different approaches that generate latent spaces from tabular data, revealing that our method improves the state-of-the-art in mapping performance. Specifically, the *Random Triplet Accuracy* highlights the capability of CP-ILS to create an effective neighborhood-preserving latent space. However, *Outlier Preservation* remains a challenging aspect, though CP-ILS still performs competitively against other methods.

Conditioning the latent space with black-box predictions facilitates the identification of a latent prediction

TABLE 4. Prototype metrics. Best results are highlighted in bold, second best are underlined.

Dataset	1-KNN Accuracy				Fidelity			
	CP-ILS	P-Sel	P-DASH	MMD	CP-ILS	P-Sel	P-DASH	MMD
adult/SVM	.9432	.8065	.8348	.8832	<u>.8312</u>	.8065	.8023	.8726
compas/RF	.9218	<u>.9149</u>	.8648	.8296	<u>.8804</u>	.9093	.8233	.8377
german/RF	<u>.8704</u>	.8406	.8453	.8705	.8921	.7796	.8377	<u>.8885</u>
diva/XGB	.7763	.7670	.6386	<u>.7740</u>	.8301	<u>.8059</u>	.7524	.7573

**FIGURE 6. Evaluation of 1-KNN Accuracy and Fidelity of prototype explanations while varying the number of prototypes on the explanation set. On the top, we have the *german* dataset when using XGB black-box. On the bottom, we have the *diva* dataset when using NN black-box.**

direction, which can be exploited to search for counterfactual explanations more efficiently. By assessing *Proximity* and *Diversity*, we demonstrate that it is possible to identify a variety of counterfactuals that are much closer to the input instances than those found by other approaches. Additionally, incorporating classifier predictions into the latent space creation helps reducing the occurrence of adversarial samples, as data with similar predictions are mapped closer together. By assessing *Plausibility* and *Robustness* of the generated counterfactuals, our intuition is confirmed that this approach leads to less adversarial explanations. However, counterfactual metrics perform differently as the latent space dimension changes, indicating that selecting the dimension solely based on the KNN criterion is insufficient for achieving optimal metrics overall. In realistic applications, this would necessitate additional time for exhaustive parameter tuning.

Finally, our method obtains remarkable results in generating high-quality prototype explanations. By effectively mapping data into a latent space conditioned by black-box predictions, CP-ILS enables the extraction of representative prototypes by using KMeans clustering, facilitating the understanding of black-box behavior. We assess the quality of these prototypes by demonstrating their resemblance to the input data and their alignment with black-box outputs. Evaluating *1-KNN Accuracy* reveals that the neighborhood preservation property facilitates the classification of data points based on their local prototypes. Additionally, by assessing *Fidelity*, we confirm the feasibility of explaining black-box decisions through latent examples. However, the chosen number of clusters or prototypes using KMeans centroids could have an impact on final explanations and

cannot be chosen blindly. Also, depending on the specific context, extracting hierarchical clusters might be more advantageous, providing a multi-scale view of the data and further enriching the interpretability of the explanations.

V. CONCLUSION

In this paper, we have presented CP-ILS, a counterfactual and prototype explanation method that exploits the similarity of data represented in a newly defined transparent latent space. CP-ILS generates explanations as Counterfactual/Prototype pairs, providing different views for understanding of the black-box internal reasoning. We have compared CP-ILS with several approaches that produce similar types of latent space, showing that our proposal improves the state-of-the-art representation performance. In particular, introducing black-box labels into the latent space, it helps to create a linear latent space that can be used to create counterfactuals and prototype explanations for tabular data. We have observed superior performance on several metrics assessing the quality of the latent space and the goodness of the counterfactual and prototype explanations obtained. By simultaneously examining prototypes and counterfactuals, humans can better understand the strengths and limitations of the classifier, identify areas where it may be biased or misaligned with real-world data, and improve its performance through targeted adjustments.

Future research directions include expanding the method to multi-class tabular datasets. The use of multiple labels would result in a latent space with multiple classification directions, one for each label. In addition, we would like to study the effects of our latent space when using image data.

Indeed, from a particular point of view, image data are more complex than tabular data and require more complex mapping to encode. However, from some preliminary experiments, it seems complex to maintain the transparency required to apply Lagrange optimization. Therefore, it is necessary to study a new ad-hoc optimization method for this type of data.

REFERENCES

- [1] J. Bennett and S. Lanning, "The Netflix prize," in *Proc. KDD Cup Workshop*, 2007, p. 35.
- [2] K. Funkhouser, "Paving the road ahead: Autonomous vehicles, products liability, and the need for a new approach," *Utah L. Rev.*, vol. 2, p. 437, Jun. 2013.
- [3] T. Miller, "Explanation in artificial intelligence: Insights from the social sciences," *Artif. Intell.*, vol. 267, pp. 1–38, Feb. 2019.
- [4] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Comput. Surveys*, vol. 51, no. 5, pp. 1–42, Sep. 2019.
- [5] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [6] F. Bodria, F. Giannotti, R. Guidotti, F. Naretto, D. Pedreschi, and S. Rinzivillo, "Benchmarking and survey of explanation methods for black box models," *Data Mining Knowl. Discovery*, vol. 37, no. 5, pp. 1719–1778, Sep. 2023.
- [7] R. Guidotti, A. Monreale, S. Ruggieri, F. Naretto, F. Turini, D. Pedreschi, and F. Giannotti, "Stable and actionable explanations of black-box models through factual and counterfactual rules," *Data Mining Knowl. Discovery*, vol. 38, no. 5, pp. 2825–2862, Sep. 2024.
- [8] R. M. J. Byrne, "Counterfactual thinking: From logic to morality," *Current Directions Psychol. Sci.*, vol. 26, no. 4, pp. 314–322, Aug. 2017.
- [9] R. Guidotti, "Counterfactual explanations and how to find them: Literature review and benchmarking," *Data Mining Knowl. Discovery*, vol. 38, no. 5, pp. 2770–2824, Sep. 2024.
- [10] T. Freiesleben, "The intriguing relation between counterfactual explanations and adversarial examples," *Minds Mach.*, vol. 32, no. 1, pp. 77–109, Mar. 2022.
- [11] V. Ballet, J. Aigrain, T. Laugel, P. Frossard, and M. Detyniecki, "Imperceptible adversarial attacks on tabular data," in *Proc. NeurIPS Workshop Robust AI Financial Services*, 2019, pp. 1–20.
- [12] S. Sharma, J. Henderson, and J. Ghosh, "CERTIFAI: Counterfactual explanations for robustness, transparency, interpretability, and fairness of artificial intelligence models," 2019, *arXiv:1905.07857*.
- [13] E. H. Rosch, "Natural categories," *Cognit. Psychol.*, vol. 4, no. 3, pp. 328–350, 1973.
- [14] J. Bien and R. Tibshirani, "Prototype selection for interpretable classification," *Ann. Appl. Statist.*, vol. 5, no. 4, pp. 2403–2424, Dec. 2011.
- [15] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 1–12.
- [16] H. Dai, B. Dai, and L. Song, "Discriminative embeddings of latent variable models for structured data," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 2702–2711.
- [17] A. Ng, "Sparse autoencoder," *CS294A Lect. Notes*, vol. 72, no. 2011, pp. 1–19, Jan. 2011.
- [18] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–20.
- [19] L. Theis, A. van den Oord, and M. Bethge, "A note on the evaluation of generative models," in *Proc. Int. Conf. Learn. Represent.*, Apr. 2016, pp. 1–10.
- [20] M. Wattenberg, F. Viégas, and I. Johnson, "How to use t-SNE effectively," *Distill*, vol. 1, no. 10, p. 2, Oct. 2016.
- [21] M. Curi, G. A. Converse, J. Hajewski, and S. Oliveira, "Interpretable variational autoencoders for cognitive models," in *Proc. IJCNN*, 2019, pp. 1–8.
- [22] S. K. Sajja, S. Mukherjee, and S. Dwivedi, "Semi-supervised counterfactual explanations," *arXiv:2303.12634*, 2023.
- [23] S. Khorram and L. Fuxin, "Cycle-consistent counterfactuals by latent transformations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 10193–10202.
- [24] F. Bodria, R. Guidotti, F. Giannotti, and D. Pedreschi, "Transparent latent space counterfactual explanations for tabular data," in *Proc. IEEE 9th Int. Conf. Data Sci. Adv. Analytics (DSAA)*, Oct. 2022, pp. 1–10.
- [25] F. Bodria, "Understanding and exploiting the latent space to improve machine learning models explainability," Ph.D. dissertation, Scuola Normale Superiore, Pisa, Italy, 2023. [Online]. Available: <https://ricerca.sns.it/handle/11384/136464?mode=full>
- [26] Y. Pu, Z. Gan, R. Henao, X. Yuan, C. Li, A. Stevens, and L. Carin, "Variational autoencoder for deep learning of images, labels and captions," in *Proc. NIPS*, 2016, pp. 2352–2360.
- [27] S. Mukherjee, H. Asnani, E. Lin, and S. Kannan, "ClusterGAN: Latent space clustering in generative adversarial networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 4610–4617.
- [28] H. Abdi and L. J. Williams, "Principal component analysis," *Wiley Interdiscipl. Rev., Comput. Statist.*, vol. 2, no. 4, pp. 433–459, Jul. 2010.
- [29] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 1–20, 2008.
- [30] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," 2018, *arXiv:1802.03426*.
- [31] E. Amid and M. K. Warmuth, "TriMap: Large-scale dimensionality reduction using triplets," 2019, *arXiv:1910.00204*.
- [32] B. Barr, M. R. Harrington, S. Sharpe, and C. Bayan Bruss, "Counterfactual explanations via latent space projection and interpolation," 2021, *arXiv:2112.00890*.
- [33] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2015, pp. 3483–3491.
- [34] H. Gm, M. K. Gourisaria, M. Pandey, and S. S. Rautaray, "A comprehensive survey and analysis of generative models in machine learning," *Comput. Sci. Rev.*, vol. 38, Nov. 2020, Art. no. 100285.
- [35] A. Nazábal, P. M. Olmos, Z. Ghahramani, and I. Valera, "Handling incomplete heterogeneous data using VAEs," *Pattern Recognit.*, vol. 107, Nov. 2020, Art. no. 107501.
- [36] M. Pawelczyk, K. Broelemann, and G. Kasneci, "On counterfactual explanations under predictive multiplicity," *UAI*, vol. 124, pp. 809–818, May 2020.
- [37] S. Joshi, O. Koyejo, W. Vijitbenjaronk, B. Kim, and J. Ghosh, "Towards realistic individual recourse and actionable explanations in black-box decision making systems," 2019, *arXiv:1907.09615*.
- [38] D. Mahajan, C. Tan, and A. Sharma, "Preserving causal constraints in counterfactual explanations for machine learning classifiers," 2019, *arXiv:1912.03277*.
- [39] D. Danks and C. Yau, "Basisvae: Interpretable simultaneous dimensionality reduction and feature-level clustering with derivative-based variational autoencoders," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 2410–2420.
- [40] C. Molnar, *Interpretable Machine Learning*, 2020. [Online]. Available: <https://leanpub.com/interpretable-machine-learning>
- [41] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *SSRN Electron. J.*, p. 841, 2017.
- [42] R. K. Mothilal, A. Sharma, and C. Tan, "Explaining machine learning classifiers through diverse counterfactual explanations," in *Proc. FAT*, 2020, pp. 607–617.
- [43] T. Laugel, M.-J. Lesot, C. Marsala, X. Renard, and M. Detyniecki, "Comparison-based inverse classification for interpretability in machine learning," in *Communications in Computer and Information Science*. Cham, Switzerland: Springer, 2018, pp. 100–111.
- [44] R. Guidotti, A. Monreale, F. Giannotti, D. Pedreschi, S. Ruggieri, and F. Turini, "Factual and counterfactual explanations for black box decision making," *IEEE Intell. Syst.*, vol. 34, no. 6, pp. 14–23, Nov. 2019.
- [45] S. Rathi, "Generating counterfactual and contrastive explanations using SHAP," 2019, *arXiv:1906.09293*.
- [46] A. White and A. S. d'Avila Garcez, "Measurable counterfactual local explanations for any classifier," in *Frontiers in Artificial Intelligence and Applications*, Pp. 2529–2535, vol. 325. Amsterdam, The Netherlands: IOS Press, 2020, pp. 2529–2535.
- [47] A. V. Loooveren and J. Klaise, "Interpretable counterfactual explanations guided by prototypes," *ECML/PKDD*, vol. 12976, pp. 650–665, May 2021.
- [48] A. Dhurandhar, P. Chen, R. Luss, C. Tu, P. Ting, K. Shanmugam, and P. Das, "Explanations based on the missing: Towards contrastive explanations with pertinent negatives," in *Proc. NeurIPS*, 2018, pp. 590–601.

- [49] R. Balasubramanian, S. Sharpe, B. Barr, J. Wittenbach, and C. Bayan Bruss, "Latent-CF: A simple baseline for reverse counterfactual explanations," 2020, *arXiv:2012.09301*.
- [50] P. Rodríguez, M. Caccia, A. Lacoste, L. Zamparo, I. Laradji, L. Charlin, and D. Vazquez, "Beyond trivial counterfactual explanations with diverse valuable explanations," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 1036–1045.
- [51] R. Guidotti, A. Monreale, S. Matwin, and D. Pedreschi, "Explaining image classifiers generating exemplars and counter-exemplars from latent representations," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 13665–13668.
- [52] C. Russell, "Efficient search for diverse coherent explanations," in *Proc. FAT*, 2019, pp. 20–28.
- [53] K. Kanamori, T. Takagi, K. Kobayashi, and H. Arimura, "DACE: Distribution-aware counterfactual explanation by mixed-integer linear optimization," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, Jul. 2020, pp. 2855–2862.
- [54] A. Artelt, V. Vaquet, R. Velicoglu, F. Hinder, J. Brinkrolf, M. Schilling, and B. Hammer, "Evaluating robustness of counterfactual explanations," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2021, pp. 01–09.
- [55] M. Pawelczyk, T. Datta, J. Van den Heuvel, G. Kasneci, and H. Lakkaraju, "Probabilistically robust recourse: Navigating the trade-offs between costs and robustness in algorithmic recourse," in *Proc. 11th Int. Conf. Learn. Represent.*, 2022, pp. 1–20.
- [56] B. Ustun, A. Spangher, and Y. Liu, "Actionable recourse in linear classification," in *Proc. FAT*, 2019, pp. 10–19.
- [57] B. Kim, O. Koyejo, and R. Khanna, "Examples are not enough, learn to criticize! criticism for interpretability," in *Proc. NIPS*, 2016, pp. 2280–2288.
- [58] P. Hase, C. Chen, O. Li, and C. Rudin, "Interpretable image recognition with hierarchical prototypes," in *Proc. AAAI Conf. Hum. Comput. Crowdsourcing*, vol. 7, 2019, pp. 32–40.
- [59] K. S. Gurumoorthy, A. Dhurandhar, G. Cecchi, and C. Aggarwal, "Efficient data representation by selecting prototypes with importance weights," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2019, pp. 260–269.
- [60] Y. Ming, P. Xu, H. Qu, and L. Ren, "Interpretable and steerable sequence learning via prototypes," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 903–913.
- [61] C. Chen, O. Li, D. Tao, A. Barnett, C. Rudin, and J. K. Su, "This looks like that: Deep learning for interpretable image recognition," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–2.
- [62] Y. Yu, Z. Ji, J. Guo, and Z. Zhang, "Zero-shot learning via latent space encoding," *IEEE Trans. Cybern.*, vol. 49, no. 10, pp. 3755–3766, Oct. 2019.
- [63] J. Kim, T.-H. Oh, S. Lee, F. Pan, and I. S. Kweon, "Variational prototyping-encoder: One-shot learning with prototypical images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 9454–9462.
- [64] T. Chen, Y. Zhang, X. Huo, S. Wu, Y. Xu, and H. S. Wong, "SphericGAN: Semi-supervised hyper-spherical generative adversarial networks for fine-grained image synthesis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 9991–10000.
- [65] W. Barbakh and C. Fyfe, "Online clustering algorithms," *Int. J. Neural Syst.*, vol. 18, no. 3, pp. 185–194, 2008.
- [66] B. Pang, T. Han, E. Nijkamp, S.-C. Zhu, and Y. N. Wu, "Learning latent space energy-based prior model," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 21994–22008.
- [67] J. Liu, K. Song, M. Feng, Y. Yan, Z. Tu, and L. Zhu, "Semi-supervised anomaly detection with dual prototypes autoencoder for industrial surface inspection," *Opt. Lasers Eng.*, vol. 136, Jan. 2021, Art. no. 106324.
- [68] D. Abati, A. Porrello, S. Calderara, and R. Cucchiara, "Latent space autoregression for novelty detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 481–490.
- [69] C. Su, S. Zhang, F. Yang, G. Zhang, Q. Tian, W. Gao, and L. S. Davis, "Attributes driven tracklet-to-tracklet person re-identification using latent prototypes space mapping," *Pattern Recognit.*, vol. 66, pp. 4–15, Jun. 2017.
- [70] K. Guu, T. B. Hashimoto, Y. Oren, and P. Liang, "Generating sentences by editing prototypes," *Trans. Assoc. Comput. Linguistics*, vol. 6, pp. 437–450, Dec. 2018.
- [71] R. Guidotti, A. Monreale, S. Matwin, and D. Pedreschi, "Black box explanation by learning image exemplars in the latent feature space," in *Lecture Notes in Computer Science*. Springer, 2020, pp. 189–205.
- [72] R. Poyiadzi, K. Sokol, R. Santos-Rodriguez, T. De Bie, and P. Flach, "FACE: Feasible and actionable counterfactual explanations," in *Proc. AAAI/ACM Conf. AI, Ethics, Soc.*, Feb. 2020, pp. 344–350.
- [73] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.
- [74] P. Biecek, "Dalex: Explainers for complex predictive models in R," *J. Mach. Learn. Res.*, vol. 19, no. 1, pp. 3245–3249, 2018.
- [75] P. D. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, 2015, pp. 1–9.
- [76] N. Ahmed, T. Natarajan, and K. R. Rao, "Discrete cosine transform," *IEEE Trans. Comput.*, vol. C-23, no. 1, pp. 90–93, May 1974.
- [77] B. Waggener, W. N. Waggener, and W. M. Waggener, *Pulse Code Modulation Techniques*. New York, NY, USA: Springer, 1995.
- [78] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.
- [79] K. Mai Ngoc and M. Hwang, "Finding the best k for the dimension of the latent space in autoencoders," in *Proc. Int. Conf. Comput. Collective Intell.*, 2020, pp. 453–464.
- [80] E. Fix and J. L. Hodges, "Discriminatory Analysis. Nonparametric discrimination: Consistency properties," *Int. Stat. Rev. Revue Internationale de Statistique*, vol. 57, no. 3, p. 238, Dec. 1989.
- [81] G. C. Linderman and S. Steinerberger, "Clustering with t-SNE, provably," *SIAM J. Math. Data Sci.*, vol. 1, no. 2, pp. 313–332, Jan. 2019.
- [82] R. T. Rockafellar, "Lagrange multipliers and optimality," *SIAM Rev.*, vol. 35, no. 2, pp. 183–238, Jun. 1993.
- [83] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. KDD*, 2016, pp. 1–20.
- [84] D. Arthur and S. Vassilvitskii, "K-means++ the advantages of careful seeding," in *Proc. 18th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2007, pp. 1027–1035.
- [85] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proc. ACM SIGKDD*, vol. 2, pp. 785–794, May 2016.
- [86] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, Oct. 2001.
- [87] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Adv. Large Margin Classifiers*, vol. 10, no. 3, pp. 61–74, Mar. 1999.
- [88] Y. Wang, "Understanding how dimension reduction tools work: An empirical approach to deciphering t-sne, umap, trimap, and pacmap for data visualization," *J. Mach. Learn. Res.*, vol. 22, p. 201, Apr. 2021.
- [89] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discovery Data*, vol. 6, no. 1, pp. 1–39, Mar. 2012.
- [90] J. B. Tenenbaum, V. D. Silva, and J. C. Langford, "A global geometric framework for nonlinear dimensionality reduction," *Science*, vol. 290, no. 5500, pp. 2319–2323, Dec. 2000.
- [91] G. C. Linderman, M. Rachh, J. G. Hoskins, S. Steinerberger, and Y. Kluger, "Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data," *Nature Methods*, vol. 16, no. 3, pp. 243–245, Mar. 2019.
- [92] L. van der Maaten, "Accelerating t-SNE using tree-based algorithms," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 3221–3245, Oct. 2014.
- [93] D. Alvarez-Melis and T. S. Jaakkola, "Towards robust interpretability with self-explaining neural networks," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 7786–7795.



SIMONE PIAGGESE received the bachelor's degree in physics and the master's degree in physics of complex systems from the University of Turin, in 2014 and 2017, respectively, and the Ph.D. degree in data science and computation from the University of Bologna, Italy, in 2023. During the Ph.D., he collaborated with the ISI Foundation in Turin, conducting research in the area of complex network science. He is currently a Postdoctoral Research Fellow with the Computer Science Department, University of Pisa, Italy, and a member of the Knowledge Discovery and Data Mining Laboratory (KDDLab), a Joint Research Group with the Information Science and Technology Institute of the National Research Council, Pisa. His research interests comprise exploring the many intersections between machine learning and complex systems. He is focusing on the explainability of learning models associated with latent spaces.



FRANCESCO BODRIA received the bachelor's degree in physics from the University of Parma, and the master's degree in physics of complex systems from the University of Turin, in 2019. He is currently a former Ph.D. Fellow in data science in the joint program offered by Scuola Normale Superiore, Scuola Superiore Sant'Anna, University of Pisa, IMT Lucca, and National Research Council. His research interests include explainable AI algorithms and in particular interpretable-by-design methods.



FOSCA GIANNOTTI is currently a Full Professor with Scuola Normale Superiore, Pisa, Italy. She is a Pioneering Scientist in mobility data mining, social network analysis, and privacy-preserving data mining. She co-leads the Pisa KDD Laboratory-Knowledge Discovery and Data Mining Laboratory, a Joint Research Initiative of the University of Pisa and ISTI-CNR, founded in 1994, as one of the earliest research laboratories on data mining. She has coordinated tens of European projects and industrial collaborations. She is the former Coordinator of SoBigData, the European research infrastructure on big data analytics and social mining, an ecosystem of ten cutting edge European research centers providing an open platform for interdisciplinary data science and data-driven innovation. Her research focus is on social mining from big data: smart cities, human dynamics, social and economic networks, ethics and trust, and diffusion of innovations. Recently, she became a recipient of a prestigious ERC Advanced Grant entitled XAI-Science and Technology for the explanation of AI decision making.



RICCARDO GUIDOTTI received the B.S. and M.S. degrees (cum laude) and the Ph.D. degree in computer science from the University of Pisa, in 2010 and 2013, respectively. His Ph.D. thesis is on Personal Data Analytics. He is currently an Assistant Professor (RTD-B) with the Department of Computer Science, University of Pisa, Italy, and a member of the Knowledge Discovery and Data Mining Laboratory (KDDLab), a Joint Research Group with the Information Science and



DINO PEDRESCHI is currently a Full Professor of computer science with the University of Pisa, and a Pioneering Scientist in mobility data mining, social network mining, and privacy-preserving data mining. He co-leads the Pisa KDD Laboratory-Knowledge Discovery and Data Mining Laboratory, a Joint Research Initiative of the University of Pisa, and the Information Science and Technology Institute of the Italian National Research Council, one of the earliest research laboratories centered on data mining. His research focus is on big data analytics and mining and their impact on society. He is a founder of the Business Informatics M.Sc. program at the University of Pisa, a course targeted at the education of interdisciplinary data scientists. He was a Visiting Scientist with Barabasi Laboratory (Center for Complex Network Research) of Northeastern University, Boston, from 2009 to 2010, and the University of Texas at Austin, from 1989 to 1990, CWI Amsterdam, in 1993, and UCLA, in 1995. In 2009, he received a Google Research Award for his research on privacy-preserving data mining.

Technology Institute of the National Research Council, Pisa. His research interests include XAI, interpretable ML, personal data analytics, time series analytics, clustering, and quantum computing. In 2015, he won the IBM Fellowship Program, in 2018, the DSAA New Generation Data Scientist Award, in 2021, and the Marco Somalvico Award for young researchers in AI.

...

Open Access funding provided by 'Università di Pisa' within the CRUI CARE Agreement